

Introducción

- Algunos modelos, como los árboles de decisión, asumen que hay una única respuesta a cada instancia
- Para algunos problemas es más interesante tener decisiones difusas, usando distribuciones de probabilidad para representar los conceptos
- ¿Se puede inyectar el conocimiento previo en la solución que se construya?

Aprendizaje bayesiano

- Cada caso de entrenamiento cambia la probabilidad estimada de que una hipótesis sea correcta (soporta ruido).
- Pueden clasificar nuevas instancias combinando distintas hipótesis.
- Son competitivos con otros métodos conocidos
- Algunos algoritmos tienen un costo alto.
- Permiten caracterizar otros métodos que no utilizan la probabilidad de forma explícita

Teorema de Bayes

$$P(A|B).P(B) = P(B|A).P(A)$$

Si consideramos un conjunto de instancias D y un hipótesis h :

$$P(h|D) = \frac{P(D|h).P(h)}{P(D)}$$

- $P(h)$ la puedo estimar o es conocimiento previo, es la probabilidad *a priori* de h
- $P(D)$ es la probabilidad previa de que se observe D y es independiente de h
- $P(D|h)$ a la probabilidad de observar datos D dado que se cumple la hipótesis h
- $P(h|D)$ es la probabilidad *a posteriori* de h luego de conocer los datos D

Teorema de Bayes

- Nos interesa la hipótesis h_{MAP} , aquella que es más probable luego de conocer los datos:

$$\begin{aligned} h_{MAP} &= \operatorname{argmax}_{h \in H} P(h|D) \\ &= \operatorname{argmax}_{h \in H} \frac{P(D|h) \cdot P(h)}{P(D)} \\ &= \operatorname{argmax}_{h \in H} P(D|h) \cdot P(h) \end{aligned}$$

- La hipótesis h_{ml} asume, además, que las hipótesis son equiprobables *a priori*:

$$h_{ml} = \operatorname{argmax}_{h \in H} P(D|h)$$

Teorema de Bayes - Ejemplo

- Un test de dopaje tiene un 77% de resultados positivos cuando hay dopaje y 10% cuando no lo hay. Se sabe que 1 de cada 100 deportistas recurre a drogas de desempeño.

$$P(dopping) = 0.01$$

$$P(\neg dopping) = 0.99$$

$$P(test \oplus | dopping) = 0.77$$

$$P(test \otimes | dopping) = 0.23$$

$$P(test \oplus | \neg dopping) = 0.10$$

$$P(test \otimes | \neg dopping) = 0.90$$

- Si el test da positivo: ¿deberíamos suspender al deportista?

$$P(test \oplus | dopping) * P(dopping) = 0.77 * 0.01 = 0.0077$$

$$P(test \oplus | \neg dopping) * P(\neg dopping) = 0.10 * 0.990 = 0.099$$

- El diagnóstico más probable es negativo, con una seguridad del 93%

Algoritmos MAP

- ¿Y si calculamos hipótesis MAP para aprendizaje conceptual?

Algoritmo list-then-eliminate - versión Bayes

Para cada hipótesis $h \in H$:

$$\text{Calculamos } \frac{P(D|h) \cdot P(h)}{P(D)}$$

Devolver $h_{MAP} = \operatorname{argmax}_{h \in H} P(h|D)$

- [Un déjà-vu]



Algoritmos MAP

- Veamos algunas de esas probabilidades
- Supongamos que:
 - el concepto objetivo pertenece a H
 - las hipótesis son equiprobables a priori, $P(h) = |H|^{-1}$
 - el conjunto D no tiene ruido:

$$P(D|h) = \begin{cases} 1 & \text{si } d_i = h(x_i), \forall d_i \in D \\ 0 & \text{en caso contrario} \end{cases}$$

- En otras palabras, evaluamos en 1 si la hipótesis es consistente con D y 0 en caso contrario: si no hay datos con ruido, h debe clasificar perfectamente a D .

Algoritmos MAP

- Bajo estos supuestos, ¿cómo estimamos $P(D)$?

$$P(D) = \sum_{h \in H} P(D|h) \cdot P(h) = \sum_{h \in VS_{H,D}} 1 \cdot P(h) + \sum_{h \notin VS_{H,D}} 0 \cdot P(h) = \frac{|VS_{H,D}|}{|H|}$$

- Si considero una hipótesis inconsistente con los datos, su probabilidad *a posteriori*:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)} = \frac{0 \cdot P(h)}{P(D)} = 0$$

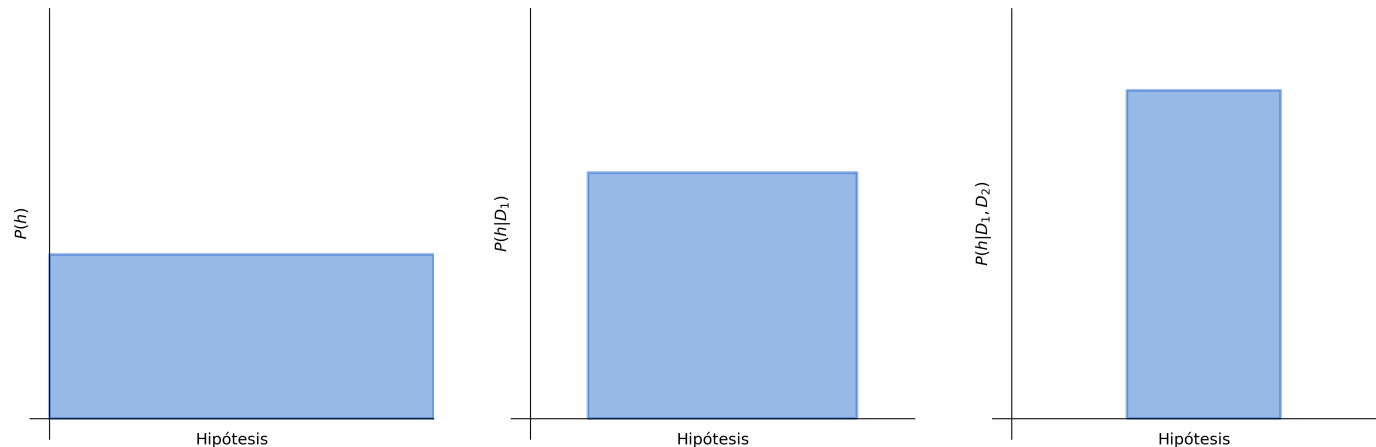
- En cambio, si es consistente:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)} = \frac{1 \cdot |H|^{-1}}{|VS_{H,D}| \cdot |H|^{-1}}$$

- Bajo estos supuestos, toda hipótesis de $VS_{H,D}$ es MAP

Algoritmos MAP

- A medida que agregos datos al entrenamiento, se reduce $V S_{H,D}$ y aumenta la probabilidad de cada hipótesis consistente



- Todo algoritmo consistente da como resultado una hipótesis MAP, si se cumple a priori una distribución uniforme sobre H
- Find-S y Candidate-Elimination no manejan ningún tipo de probabilidad y sin embargo son algoritmos MAP

Algoritmos MAP

- ¿Existen otras condiciones bajo las cuales Find-S sea un algoritmo MAP?
- Sí, por ejemplo cuando la distribución sobre H asigna más probabilidad a las hipótesis más específicas y no hay ruido en la entrada.
- Podemos caracterizar los algoritmos aun cuando éstos no utilizan explícitamente probabilidades: debemos dar distribuciones $P(D|h)$ y $P(h)$ bajo las cuales los algoritmos construyen hipótesis MAP.
- Esta es una alternativa al sesgo para caracterizar los supuestos bajo los cuales un algoritmo aprende.

Clasificador bayesiano óptimo

- ¿Es la hipótesis más probable aquella que nos da la clasificación más probable de una nueva instancia?

$$P(h_1|D) = 0.4 \quad h_1(x) = \oplus$$

$$P(h_2|D) = 0.3 \quad h_2(x) = \otimes$$

$$P(h_3|D) = 0.3 \quad h_3(x) = \otimes$$

- En este ejemplo:
 - ¿qué valor devuelve la hipótesis MAP?
 - ¿cuál es la clasificación más probable de x ?

Clasificador bayesiano óptimo

- El valor más probable se obtiene combinando los resultados de todas las hipótesis:

$$P(v|D) = \sum_{h \in H} P(v|h)P(h|D)$$

- La clasificación bayesiana óptima es:

$$\operatorname{argmax}_{v \in V} \sum_{h \in H} P(v|h)P(h|D)$$

Clasificador bayesiano óptimo

- En nuestro ejemplo:

$$P(h_1|D) = 0.4 \quad P(h_2|D) = 0.4 \quad P(h_3|D) = 0.3$$

$$P(\oplus|h_1) = 1 \quad P(\oplus|h_2) = 0 \quad P(\oplus|h_3) = 0$$

$$P(\otimes|h_1) = 0 \quad P(\otimes|h_2) = 1 \quad P(\otimes|h_3) = 1$$

$$\left. \begin{aligned} P(\oplus|D) &= \sum_{h \in H} P(\oplus|h)P(h|D) = 0.4 \\ P(\otimes|D) &= \sum_{h \in H} P(\otimes|h)P(h|D) = 0.6 \end{aligned} \right\} \operatorname{argmax}_{v \in \{\oplus, \otimes\}} \sum_{h \in H} P(v|h) \cdot P(h|D) = \otimes$$

Clasificador bayesiano óptimo

- Se maximiza la probabilidad de clasificar correctamente nuevas instancias dado el conjunto de entrenamiento y las probabilidades a priori de las hipótesis
- Independientemente de su implementación, cualquier sistema que clasifique las instancias de esta manera es un CBO
- Por ejemplo, tomar el resultado de la votación entre las hipótesis de $VS_{H,D}$ es un CBO
- El clasificador que se obtiene no pertenece necesariamente al espacio H original
- El método es muy costoso: debemos estimar la probabilidad a posteriori de todas las hipótesis

Clasificador sencillo

- Consideremos instancias $x = \langle a_1, \dots, a_n \rangle$ y función objetivo $f : X \rightarrow V$
- Buscamos:

$$\begin{aligned} v_{MAP} &= \operatorname{argmax}_{v_i \in V} P(v_i | a_1, \dots, a_n) \\ &= \operatorname{argmax}_{v_i \in V} \frac{P(a_1, \dots, a_n | v_i) \cdot P(v_i)}{P(a_1, \dots, a_n)} \\ &= \operatorname{argmax}_{v_i \in V} P(a_1, \dots, a_n | v_i) \cdot P(v_i) \end{aligned}$$

- ¿Cómo estimamos esos valores a partir de D ?

Clasificador sencillo

¿Cómo se clasificaría a la instancia
< soleado, frío, alta, fuerte >?

$$P(v_i)$$

$$P(< \text{soleado, frío, alta, fuerte} > | v_i)$$

# Ej	Tiempo	Temperatura	Humedad	Viento	Juega
1	Soleado	Caluroso	Alta	Suave	No
2	Soleado	Caluroso	Alta	Fuerte	No
3	Nuboso	Caluroso	Alta	Suave	Sí
4	Lluvioso	Templado	Alta	Suave	Sí
5	Lluvioso	Frío	Normal	Suave	Sí
6	Lluvioso	Frío	Normal	Fuerte	No
7	Nuboso	Frío	Normal	Fuerte	Sí
8	Soleado	Templado	Alta	Suave	No
9	Soleado	Frío	Normal	Suave	Sí
10	Lluvioso	Templado	Normal	Suave	Sí
11	Soleado	Templado	Normal	Fuerte	Sí
12	Nuboso	Templado	Alta	Fuerte	Sí
13	Nuboso	Caluroso	Normal	Suave	Sí
14	Lluvioso	Templado	Atla	Fuerte	No

Clasificador sencillo

- Asumimos independencia de atributos (!):

$$\begin{aligned} v_{NB} &= \operatorname{argmax}_{v_i \in V} \left[P(v_i) \cdot \prod_j P(a_j | v_i) \right] \\ &= \operatorname{argmax}_{v_i \in \{sí, no\}} P(v_i) \cdot \\ &\quad P(tiempo = soleado | v_i) \cdot \\ &\quad P(temp = frío | v_i) \cdot \\ &\quad P(humedad = alta | v_i) \cdot \\ &\quad P(viento = fuerte | v_i) \end{aligned}$$

- Se estiman esas 10 probabilidades

Clasificador sencillo

$$P(juega = sí) = \frac{9}{14}$$

$$P(tiempo = soleado | juega = sí) = \frac{2}{9}$$

$$P(temp = frío | juega = sí) = \frac{3}{9}$$

$$P(humedad = alta | juega = sí) = \frac{3}{9}$$

$$P(viento = fuerte | juega = sí) = \frac{3}{9}$$

# Ej	Tiempo	Temperatura	Humedad	Viento	Juega
1	Soleado	Caluroso	Alta	Suave	No
2	Soleado	Caluroso	Alta	Fuerte	No
3	Nuboso	Caluroso	Alta	Suave	Sí
4	Lluvioso	Templado	Alta	Suave	Sí
5	Lluvioso	Frío	Normal	Suave	Sí
6	Lluvioso	Frío	Normal	Fuerte	No
7	Nuboso	Frío	Normal	Fuerte	Sí
8	Soleado	Templado	Alta	Suave	No
9	Soleado	Frío	Normal	Suave	Sí
10	Lluvioso	Templado	Normal	Suave	Sí
11	Soleado	Templado	Normal	Fuerte	Sí
12	Nuboso	Templado	Alta	Fuerte	Sí
13	Nuboso	Caluroso	Normal	Suave	Sí
14	Lluvioso	Templado	Atla	Fuerte	No

Clasificador sencillo

$$P(juega = no) = \frac{5}{14}$$

$$P(tiempo = soleado|juega = no) = \frac{3}{5}$$

$$P(temp = frío|juega = no) = \frac{1}{5}$$

$$P(humedad = alta|juega = no) = \frac{4}{5}$$

$$P(viento = fuerte|juega = no) = \frac{3}{5}$$

# Ej	Tiempo	Temperatura	Humedad	Viento	Juega
1	Soleado	Caluroso	Alta	Suave	No
2	Soleado	Caluroso	Alta	Fuerte	No
3	Nuboso	Caluroso	Alta	Suave	Sí
4	Lluvioso	Templado	Alta	Suave	Sí
5	Lluvioso	Frío	Normal	Suave	Sí
6	Lluvioso	Frío	Normal	Fuerte	No
7	Nuboso	Frío	Normal	Fuerte	Sí
8	Soleado	Templado	Alta	Suave	No
9	Soleado	Frío	Normal	Suave	Sí
10	Lluvioso	Templado	Normal	Suave	Sí
11	Soleado	Templado	Normal	Fuerte	Sí
12	Nuboso	Templado	Alta	Fuerte	Sí
13	Nuboso	Caluroso	Normal	Suave	Sí
14	Lluvioso	Templado	Atla	Fuerte	No

Clasificador sencillo

- Entonces:

$$P(ti = soleado|j = sí) \cdot P(te = frío|j = sí) \cdot P(hu = alta|j = sí) \cdot P(vi = fuerte|j = sí) \cdot P(j = sí) = 0.0053$$

$$P(ti = soleado|j = no) \cdot P(te = frío|j = no) \cdot P(hu = alta|j = no) \cdot P(vi = fuerte|j = no) \cdot P(j = no) = 0.0206$$

- La respuesta es negativa con un 79.5% de seguridad
- ¿Qué sucede con $P(tiempo = nuboso|jugar = no)$?
- ¿Cómo clasifico a $\langle nuboso, frío, alta, fuerte \rangle$?

m-estimador

- Cuando no hay ejemplos para algún caso, la estimación es nula y eso vuelve cero la productoria
- Se utiliza un m-estimador:
$$\frac{n_c + m \cdot p_c}{n + m}$$
 - n la cantidad de ejemplos
 - n_c es la ocurrencia en los n datos
 - m es el tamaño equivalente de muestra
 - p_c es una estimación *a priori*
- El m-estimador aumenta al conjunto en m ejemplos distribuidos según p
- Por ejemplo, se toma $p_c = |\text{valores}|^{-1}$

m-estimador

- ¿Qué sucede con $P(tiempo = nuboso | jugar = no)$?
- Hay cinco ejemplos negativos de los cuales ninguno es nuboso: $n = 5$ y $n_c = 0$
- Tiempo toma tres valores, elijo $p_c = \frac{1}{|valores|} = \frac{1}{3}$
- Para $m = 2$, $P(tiempo = nuboso | jugar = no) = \frac{0 + 2 * \frac{1}{3}}{5 + 2} = 0.095$

Valor	#	e	m=1	m=2	m=3
soleado	3	0.6	0.556	0.524	0.467
nuboso	0	0	0.056	0.095	0.167
lluvioso	2	0.4	0.389	0.381	0.367

Valores continuos

- Se pueden discretizar, creando rangos de valores
- Otra opción es estimar una distribución para el argumento. Por ejemplo, asumir una distribución normal y usar su densidad estimando μ_{a,v_i} y σ_{a,v_i} con los datos
- En ese caso, el valor es:

$$\frac{1}{\sqrt{2\pi\sigma_{a,v_i}^2}} e^{-\frac{(x_i - \mu_{a,v_i})^2}{2\sigma_{a,v_i}^2}}$$

Estabilidad numérica

- Las productorias pueden generar números muy pequeños e inestabilidad numérica
- Se aplica la función logaritmo:

$$\log \left(P(v_i) \prod_j P(a_j|v_i) \right) = \log(P(v_i)) + \sum_j \log(P(a_j|v_i))$$

- Como $\log$ es creciente:

$$v_{NB} = \operatorname{argmax}_{v_i \in V} \left[\log(P(v_i)) + \sum_j \log(P(a_j|v_i)) \right]$$

Clasificador bayesiano de textos

- El objetivo es clasificar textos de soporte en categorías: *pagos, soporte, envíos*, etc.
- Por ejemplo:
 - *Me dio error pero me figura el pago en mi tarjeta.* → Pagos
 - *No encuentro el producto que busco en la web.* → Soporte
 - *El producto llegó roto.* → Envíos

Clasificador bayesiano de textos

- Si cada palabra es un atributo puedo construir un clasificador con NB considerando las distintas palabras:

$$v_{NB} = \underset{c_i \in \{pagos, soporte, \dots\}}{\operatorname{argmax}} \left[P(c_i) \cdot \prod_j P(t_j = w_k | c_i) \right]$$

- Por ejemplo, para *El producto llegó roto*:

$$P(t_1 = el | pagos) \cdot P(t_2 = producto | pagos) \cdot P(t_3 = llegó | pagos) \cdot P(t_4 = roto | pagos) \cdot P(pagos) \\ P(t_1 = el | soporte) \cdot P(t_2 = producto | soporte) \cdot P(t_3 = llegó | soporte) \cdot P(t_4 = roto | soporte) \cdot P(soporte)$$

...

- Pero ¿las palabras son independientes entre sí?
- ¿Es práctico?

Clasificador bayesiano de textos

- Con un vocabulario de 50 mil palabras, textos de a los sumo 40 palabras, y 3 categorías tengo 6 millones de parámetros $P(t_j = w_k | c_i)$
- Agregamos otra susposición (irreal): la aparición de las palabras es independiente de su posición en el mensaje:

$$P(t_j = w_k | c_i) = P(w_k | c_i), \forall a_j$$

- Ahora para «El paquete llegó roto.» y «Llegó roto el paquete.» se calculan las mismas probabilidades:
- La cantidad de parámetros se reduce a 150 mil parámetros $P(w_k | c_i)$

Clasificador bayesiano de textos

- Se pueden aplicar otras simplificaciones, como limitar el vocabulario eliminando palabras vacías (*stop words*) o de acuerdo a alguna medida de relevancia (TF-IDF)
- ¿Qué sucede si en mi conjunto de entrenamiento una palabra no se da en los textos de alguna clase ?
- ¿Qué sucede si una palabra de un nuevo mensaje no se había visto antes?

Clasificador bayesiano de textos

- Estimaciones:

$$P(c_j) = \frac{|Documentos_{c_j}|}{|Documentos|}, \forall c_j \in Categorías$$

$$P(w_k|c_j) = \frac{n_{w_k,c_j} + 1}{n_{c_j} + |Vocabulario|}, \forall w_k \in Vocab., c_j \in Categorías$$

n_{c_j} es el total de palabras en los textos clasificados como c_j

n_{w_k,c_j} es el total de apariciones de w_k en textos clasificados como c_j

- Clasificación:

$$v_{NB} = \underset{c_i \in \{pagos, soporte, \dots\}}{\operatorname{argmax}} \left[P(c_i) \cdot \prod_{w_k \in (Entrada \cap Vocab.)} P(w_k|c_i) \right]$$

Principio MDL

- Volviendo a las hipótesis h_{MAP} ...
- Sabemos que $-\log(p_i)$ es el largo óptimo para codificar un mensaje m_i si su probabilidad de aparición es p_i (Shannon & Weaver)

$$\begin{aligned} h_{MAP} &= \operatorname{argmax}_{h \in H} P(D|h) \cdot P(h) \\ &= \operatorname{argmin}_{h \in H} [-\log(P(D|h)) - \log(P(h))] \end{aligned}$$

- $-\log(P(D|h))$ y $-\log(P(h))$ son los largos de codificaciones óptimas
- h_{MAP} es entonces la hipótesis más simple que explica los datos (¡la navaja de Ockham!)

Principio MDL

Principio Largo Mínimo de Descripción (MDL):

$$h_{MDL} = \operatorname{argmin}_h [\langle \langle h \rangle \rangle_{C_1} + \langle \langle D|h \rangle \rangle_{C_2}]$$

donde C_1 y C_2 son las codificaciones para las hipótesis y los datos dada la hipótesis

- Sabemos que h_{MDL} es MAP para las codificaciones óptimas
- Pero las codificaciones óptimas suelen ser desconocidas, y en la práctica suelen ya estar determinadas por el problema

Principio MDL - Árboles de decisión

- Para C_1 elegimos una codificación que asigne códigos más largos a árboles con más nodos y aristas
- ¿Qué codificación elegimos para C_2 habiendo transmitido el árbol?
 - Supongamos que los ejemplos $x_1 \dots x_n$ son conocidos por el emisor y el receptor, y lo único que se transmite es $\langle f(x_1) \dots f(x_n) \rangle$
 - Si la hipótesis clasifica perfectamente los ejemplos, no hay nada para transmitir, pero con seguridad el árbol sea más grande
 - En cambio, cuando algún ejemplo es clasificado erróneamente hay que transmitir un mensaje donde se identifica al ejemplo y a su valor correcto
- El principio MDL nos permite balancear la complejidad del árbol versus los errores que se cometen

Principio MDL - Árboles de decisión

- Las aplicaciones de este principio dieron resultados similares a los algoritmos clásicos
- ¿Podemos afirmar, entonces, que Ockham tenía razón?
- La respuesta es negativa, la justificación bayesiana sólo es válida cuando estamos ante las codificaciones óptimas
- Para determinar las codificaciones precisamos saber $P(h)$ y $P(D|h)$
- En la práctica se busca una especificación que nos parezca mejor... y aplicamos el algoritmo